OBJECT ORIENTED PROGRAMMING THROUGH JAVA

UNIT1

- **Object Oriented Programming**: Basic concepts, Principles,
- Program Structure in JAVA: Introduction, Writing Simple JAVA Programs, Elements or Tokens in JAVA Programs, JAVA Statements, Command Line Arguments, User Input to Programs, Escape Sequences, Comments, Programming Style.
- **Data Types**, **Variables, and Operators** :Introduction, Declaration of Variables, Data Types, Type Casting, Scope of Variable, Identifier, Literal Constants, Symbolic Constants, Formatted Output with printf() Method, Static Variables and Methods, Attribute Final,
- **Introduction to Operators**, Precedence and Associativity of Operators, Assignment Operator (=), Basic Arithmetic Operators, Increment (++) and Decrement (- -) Operators, Ternary Operator, Relational Operators, Boolean Logical Operators, Bitwise Logical Operators.
- **Control Statements** :Introduction, if Expression, Nested if Expressions, if–else Expressions, Ternary Operator?:, Switch Statement, Iteration Statements, while Expression, do–while Loop, for Loop, Nested for

JAVA FUNDAMENTALS

1. OBJECT ORIENTED PROGRAMMING (OOP)

A. Basic Concepts
- Class
- Object
- Method
- Message Passing

B. Principles
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

C. Advantages of OOP
- Modularity
- Reusability
- Maintainability
- Flexibility

2. PROGRAM STRUCTURE IN JAVA

1. Introduction to Java
2. Writing Simple Java Programs
3. Elements / Tokens in Java Programs
   - Keywords
   - Identifiers
   - Literals
   - Operators
   - Separators
4. Java Statements
5. Command Line Arguments
6. User Input to Programs
7. Escape Sequences
8. Comments
   - Single Line (//)
   - Multi Line (/* */)
   - Documentation (/** */)
9. Programming Style
   - Indentation
   - Naming Conventions
   - Readability
   - Documentation

3. DATA TYPES, VARIABLES AND OPERATORS

A. Data Types

1. Primitive (8 Types)
- byte
- short
- int
- long
- float
- double
- char
- boolean

2. Non-Primitive (Reference Types)
- String
- Array
- Class
- Interface

B. Variables
- Declaration
- Scope of Variable
- Type Casting
- Static Variables
- Static Methods
- final Attribute

C. Constants
- Literal Constants (e.g., 10, 3.14, 'A')
- Symbolic Constants (using final keyword)

D. Formatted Output
printf() Method

E. Operators
1. Assignment (=)
2. Arithmetic (+, −, *, /, %)
3. Increment (++)
4. Decrement (−−)
5. Ternary (? :)
6. Relational (<, >, <=, >=, ==, !=)
7. Boolean Logical (&&, ||, !)
8. Bitwise Logical (&, |, ^, ~, <<, >>, >>>)
9. Precedence & Associativity (of Operators)

4. CONTROL STATEMENTS

A. Decision Making
- if Expression
- Nested if Expressions
- if–else Expressions
- Ternary Operator (? :)
- Switch Statement

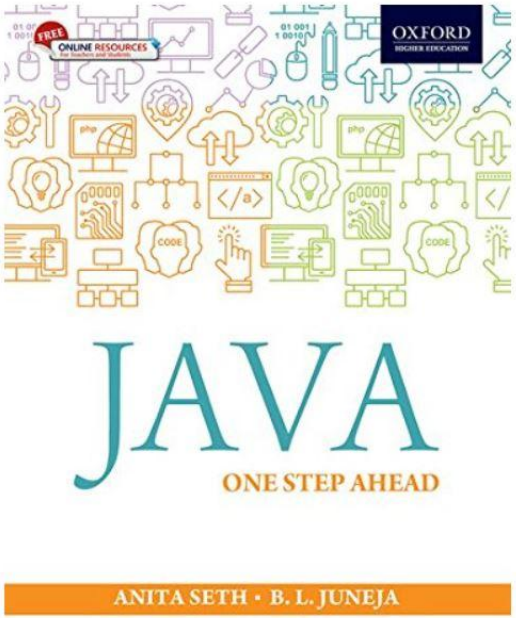
B. Iteration / Looping
- while Expression
- do–while Loop
- for Loop
- Nested for Loop
- For–Each for Loop

C. Jump Statements
- break Statement
- continue Statement

These topics form the foundation of Java Programming and are essential for building problem-solving skills using the Java language.

Text Books

- JAVA one step ahead, Anitha Seth, B.L.Juneja, Oxford.
- Joy with JAVA, Fundamentals of Object Oriented Programming, Debasis Samanta, Monalisa S[illegible]ge, 2023.



History



- **1990 –** Sun Microsystems initiated the **Stealth Project**, later known as the **Green Project**.
- **1991 –** The Green Project team was formed, consisting of **James Gosling, Patrick Naughton, Mike Sheridan, and Bill Joy**.
- **1991 – James Gosling** began developing a new programming language inspired by **C and C++**, with the primary goal of creating a **platform-independent** language.
- The language was initially named **Oak**, after an oak tree outside Gosling's office.
- Since **Oak** was already a registered trademark, it was later renamed **Java**.
- **1994 – Patrick Naughton** developed **WebRunner**, a Java-based web browser to demonstrate the capabilities of **Java applets**.
- It was later renamed **HotJava**.
- **1995 – Sun Microsystems** officially announced **Java** and the **HotJava** web browser, marking Java's public release..

Processor Developers

- *Intel*
- *AMD*
- *Qualcomm*
- *NVIDIA*
- *IBM*
- *Samsung*
- *Motorola*
- *Hewlett-Packard (hp)*
- *Dell*
- *Acer*
- *VIA*
- *Marvell*
- *Global Foundries*
- *Media Tek*
- *Atmel*
- *Freescale*
- *MSNCC-Microcell*
- *Transmeta*
- *Cyrix*
- *Rockchip*

C

Operating System	Processor	Same .exe Runs?
Windows	Intel	✔
Windows	ARM	✘ (Recompile Required)
Linux	Intel	✘ (Recompile Required)
Linux	ARM	✘ (Recompile Required)
macOS	Apple M-Series	✘ (Recompile Required)

JAVA

Operating System	Processor	Runs?
Windows	Intel	✔
Windows	ARM	✔
Linux	Intel	✔
Linux	ARM	✔
macOS	Apple M-Series	✔

History of Java

- JDK Alpha and Beta (1995)
- JDK 1.0 (23rd Jan, 1996)
- JDK 1.1 (19th Feb, 1997)
- J2SE 1.2 (8th Dec, 1998)
- J2SE 1.3 (8th May, 2000)
- J2SE 1.4 (6th Feb, 2002)
- J2SE 5.0 (30th Sep, 2004)
- Java SE 6 (11th Dec, 2006)
- Java SE 7 (28th July, 2011)
- Java SE 8 (18th March, 2014) - 217 Packages and 4240 Classes.
- Java SE 9 (September 2017)
- Java SE 10 (March 2018)
- Java SE 11 (September 2018)
- Java SE 12 (March 2019)
- Java SE 13 (September 2019)
- And so on at present JAVA SE 26 (March 17, 2026)

Why Java populated

Portability
Security
Applets

About Java

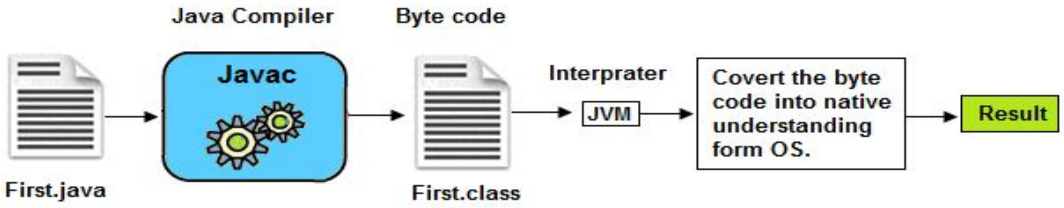
Platform	Full Form	Purpose
Java SE (formerly J2SE)	Java Platform, Standard Edition	Used for developing and deploying standalone desktop and console applications. It provides the core Java libraries and APIs.
Java EE (formerly J2EE, now Jakarta EE)	Java Platform, Enterprise Edition	Used for developing large-scale, distributed, and web-based enterprise applications such as e-commerce, banking, and ERP systems.
Java ME (formerly J2ME)	Java Platform, Micro Edition	Designed for developing Java applications for embedded systems and resource-constrained devices such as feature phones, IoT devices, and consumer electronics.

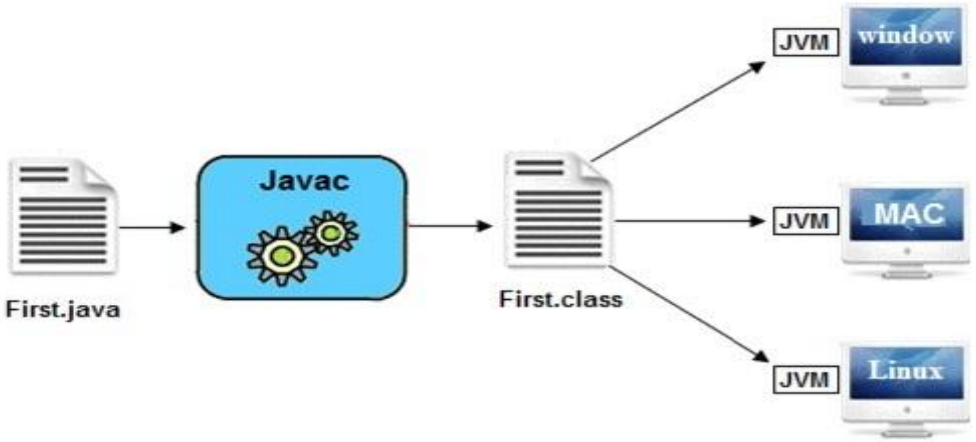
Java SE

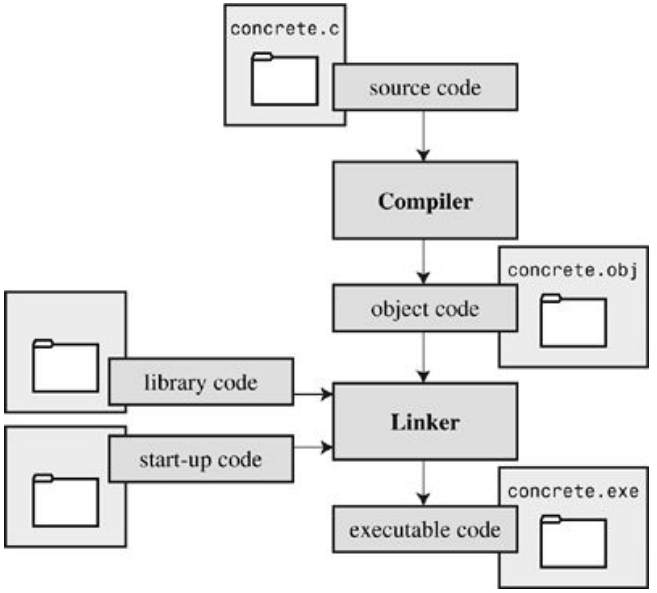
- Total Java Classes
  - 4240 in java 8
  - 4024 in java 7
  - 3793 in java 6
  - 3279 in java 5.0
- Total Java Packages
  - 217

As per SE22
Total Classes 5000+
Total Packages 430

Java's Magic : Bytecode



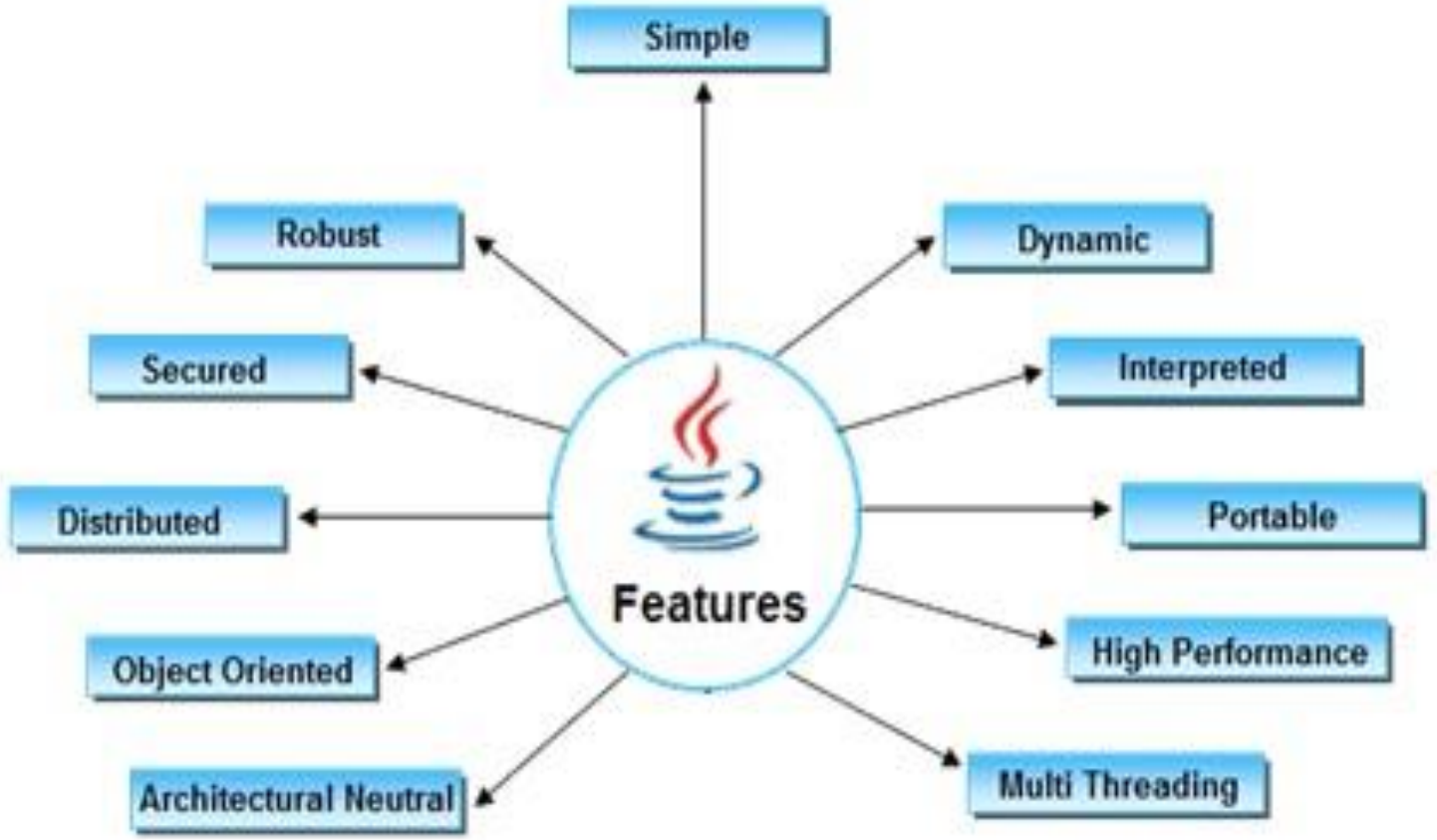




Byte code: optimized set of instructions

JIT (Just in Time Compiler)

Java Features

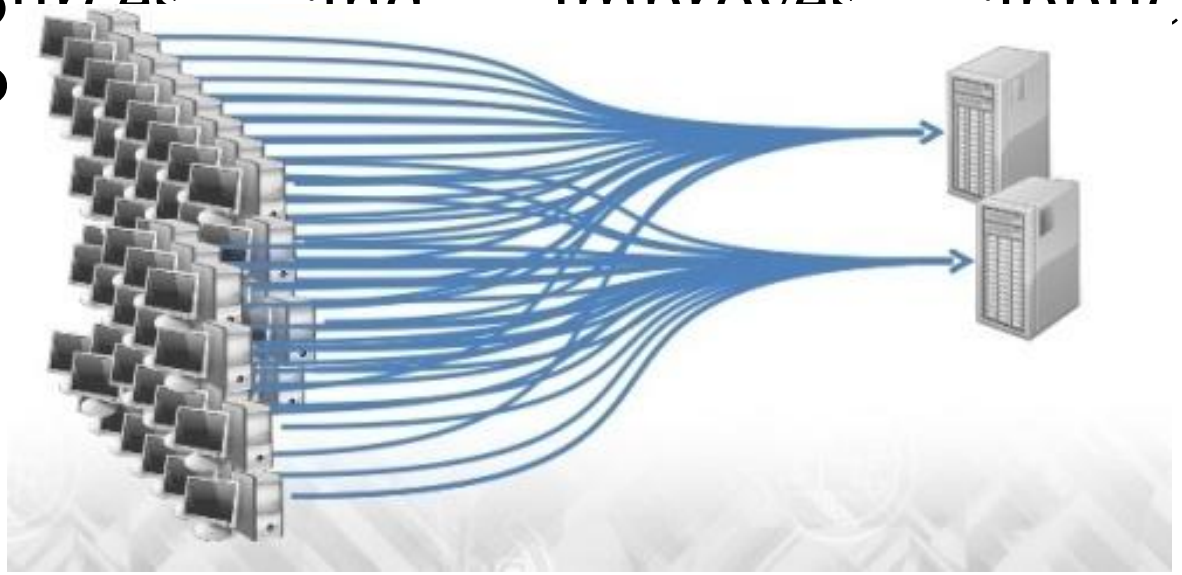


Java Features

- **Simple**
  - Java has a simple syntax that is easy to learn.
  - It removes complex features of C++, such as pointers, operator overloading, and multiple inheritance (through classes).
- **Object Oriented**
  - Java is based on the concepts of **Objects** and **Classes**.
  - It supports **Encapsulation, Inheritance, Polymorphism**, and **Abstraction**, making programs modular and reusable.
- **Robust**
  - Java is robust or strong programming language because of its capability to handle run-time errors, such as exception handling.

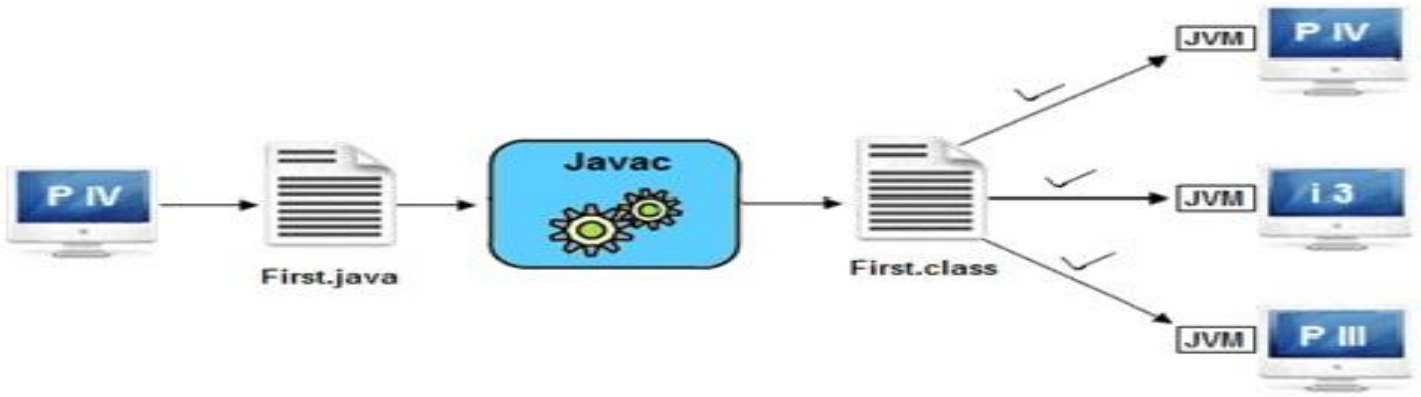
Java Features

- **Multi Threading:**
  - Java supports multiple threads executing simultaneously.
  - This allows efficient utilization of CPU resources and improves application resp



Java Features

- **Architecture-Neutral: - Hardware Independence**
  - Java source code is compiled into bytecode, which is independent of the underlying hardware architecture.
  - The same bytecode runs on any



Java Features

- **Interpreted:**
  - Java byte code is interpreted (and JIT-compiled) by the JVM.
  - This enables platform independence.

- **High Performance:**
  - The problem with interpreter inside JVM is that it is slow.
  - Because of this, Java programs used to run slow.
  - To overcome this, along with interpreter, Java people have introduced JIT (Just-in-Time) compiler, which enhances the speed of execution.

Java Features

- **Distributed:**
  - Java supports distributed computing over networks.
  - It provides APIs for networking, Remote Method Invocation (RMI), Web Services, and distributed applications.
- **Dynamic:**
  - Java supports dynamic loading of classes during program execution.
  - New classes and libraries can be loaded whenever required.

Java Features

- **Secure:**
  - It is more secured language compare to other languages; In this, program is covered into byte code after compilation which is not readable by human
  - Restricted to access other areas in the system
- **Portable:** - **Operating System Independence**
  - Java programs can run on different operating systems without modification.
  - This is possible because of the **Java Virtual Machine (JVM)**.
  - If any language supports platform independent and architectural neutral features, then that language is portable.

Platform Independence =

Hardware Independence (architectural Neutral)

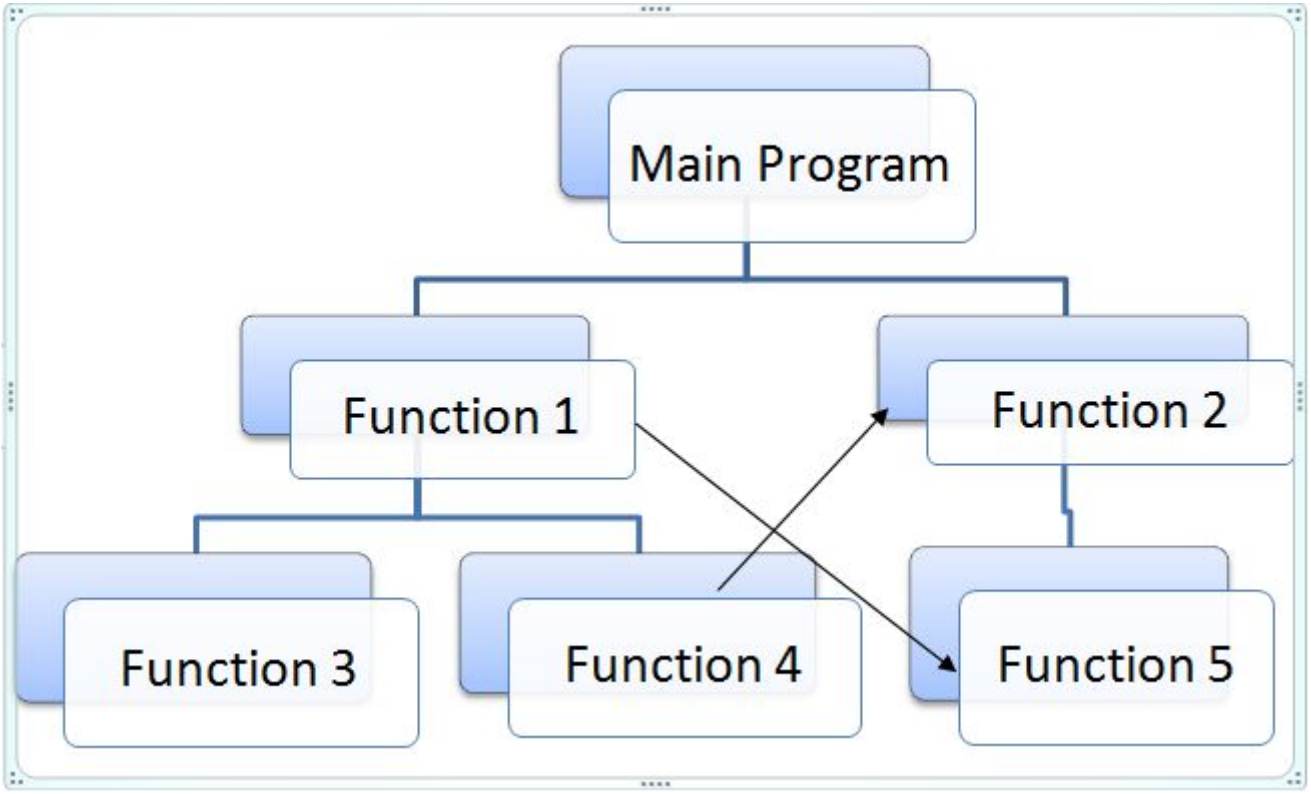
+

Portable (Operating System Independence through JVM)

Procedural/Structured Programming

- Huge programs are divided into smaller programs as functions. These functions can call one another.
- No importance is given to security of data and importance is laid on doing things.
- Data passes globally from function to function
- Most functions access global data
- Follows top-to-bottom approach

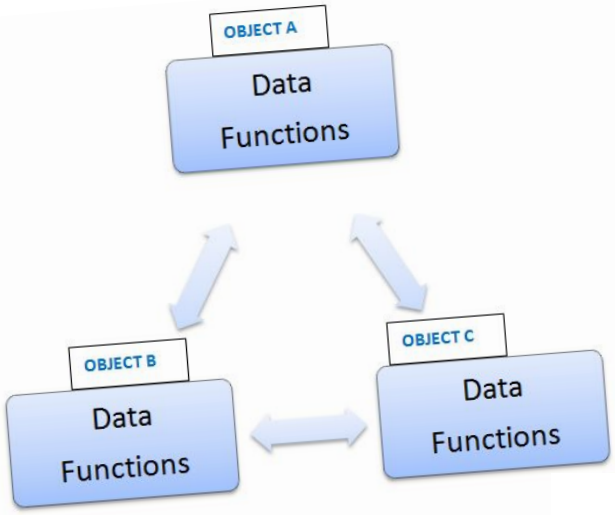
Procedural/Structured Programming



Object Oriented Programming

- *OOP* has more importance to data rather than doing things
- Programs are divided into classes .
- OOP allows bottom-up approach
- New data items and member functions can be comfortably added whenever essential.
- Data is private and prevented from external functions
- Objects can communicate with each other through methods

Object Oriented Programming



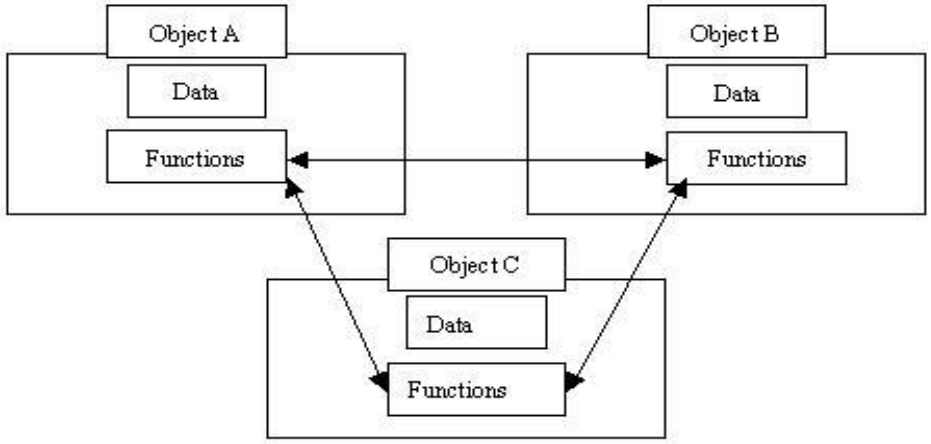


Fig 4

Difference Between POP and OOP

POP (Procedure-Oriented Programming)	FEATURE	OOP (Object-Oriented Programming)
Focuses on functions/procedures.	1. Focus	Focuses on objects and classes.
Data and functions are separate. (DATA ↔ FUNCTIONS)	2. Data & Functions	Data and functions are combined into objects. (OBJECT: DATA, FUNCTIONS)
Follows a top-down approach.	3. Approach	Follows a bottom-up approach.
Data is less secure because it is globally accessible.	4. Data Security	Data is more secure through encapsulation.
Does not support inheritance or polymorphism.	5. Key Features	Supports inheritance, polymorphism, encapsulation and abstraction.
Suitable for small programs.	6. Program Size	Suitable for large and complex programs.
Less code reusability and harder to maintain.	7. Code Reusability & Maintenance	More code reusability and easier to maintain.
Easier to write for simple tasks.	8. Ease	Easier to manage and extend in large applications.
Examples: C, Pascal	9. Examples	Examples: Java, C++, Python, C#

IN SIMPLE WORDS

POP:

The program is divided into functions that perform tasks.

Example:

Create separate functions like add(), delete(), and update().

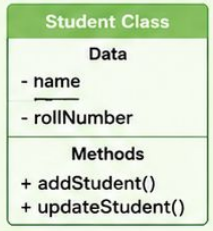


OOP:

The program is divided into objects that contain both data and functions.

Example:

Create a Student class with data (name, roll number) and methods like addStudent() and updateStudent().



Conclusion: POP is function-centered, while OOP is object-centered.

Need of OOP

- Though procedural-oriented programs are extremely powerful, they have some of the following limitations:
  - It is difficult to add any new functionality or to upgrade a program.
  - Functions have unrestricted access to global data.
  - Importance is given to the operation on data rather than the data itself.

Need of OOP

- An object-oriented approach to the application development enables us to design programs.
  - *OOP* has more importance to data rather than doing things
  - Programs are divided into classes - Modularity .
  - OOP allows bottom-up approach
  - New data items and member functions can be comfortably added whenever essential
  - Data is private and prevented from external functions
  - Objects can communicate with each other through methods

Advantages of OOP

- ***Modular structure -*** provides a clear and modular structure for program development.
- ***Easy to maintain and upgrade-*** Object-oriented systems can be easily upgraded.
- ***Inheritance - A***llows us to eliminate redundant code and use existing classes.
- ***Reusability*** - functions and modules can be reused by other users without any modification.
- ***Data hiding -***The programmer can hide the data and functions in a class from other classes.

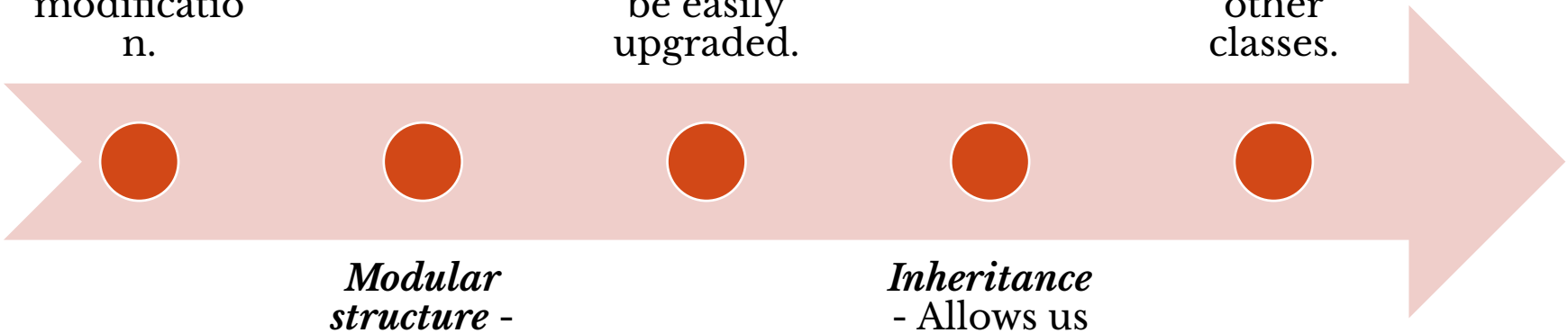
Reusability - functions and modules can be reused by other users without any modification.

Modular structure - provides a clear and modular structure for program development.

Easy to maintain and upgrade- Object-oriented systems can be easily upgraded.

Inheritance - Allows us to eliminate redundant code and use existing classes.

Data hiding -The programmer can hide the data and functions in a class from other classes.



Characteristics of Object-Oriented Programming

- Class
- Object
- Methods and Message Passing
- Data Abstraction
- Encapsulation and Data hiding
- Inheritance
- Polymorphism
- Delegation

Core Pillars of OOP:

- **Abstraction**
- **Encapsulation**
- **Inheritance**
- **Polymorphism**

Class

Object	Properties	Methods
	car.name = Verna car.model = 2017 car.weight = 1250kg car.color = white	car.start() car.drive() car.brake() car.stop()

- In OOP, a **class** is a basic building block.
- A class is a user-defined data type.
- It consists of data members and member functions (methods), which can be accessed and used by creating an instance of that class

A class is a blueprint or template that defines data (variables) and behavior (methods).

Object



- An Object is an instance of a class.
- When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.
- An object has an identity(name), state/properties, and behavior.

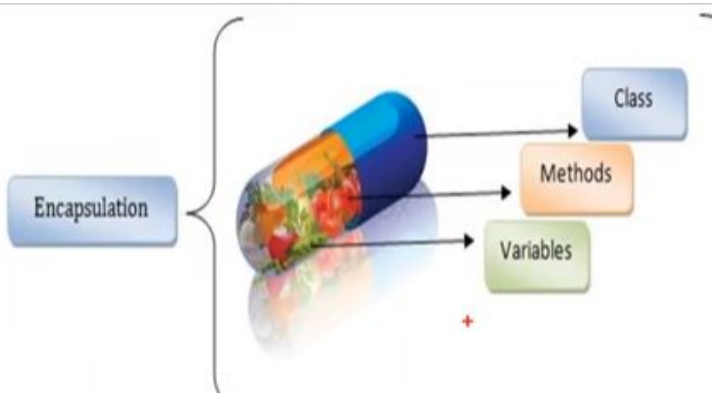
Data Abstraction



- Abstraction means showing only essential features and hiding implementation details.
- **Example:** When driving a car, you use the steering and pedals without knowing the engine's internal mechanism.

Encapsulation and Data hiding

- It is a mechanism of wrapping the **data and methods** into a single unit (i.e., class).
- The data of a class may be hidden so that it cannot be accessed by the outside code.
  - Ex: Account balance cannot be accessed directly from outside the class.



- Classes have the provision to hide the data or methods from the user.
- A class member may be declared as *public*, *protected* or *private*.
  - A member declared as private cannot be accessed by any code outside the class.
  - Only class members can access a private member.
  - The one declared protected can be accessed by the class members as well as members of the subclass derived from the class.
  - A member declared public can be accessed by any outside code.
  - Whereas the one declared without any access specifier is accessible to classes of the same package only.

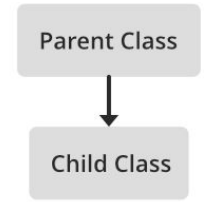
Access Specifiers

- Public
- Private
- Protected
- Default

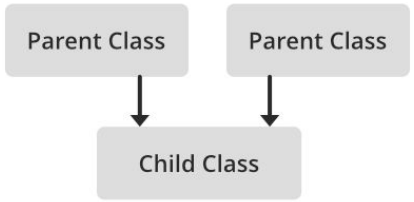
Applied to variables, constructors, methods, and classes.

Inheritance

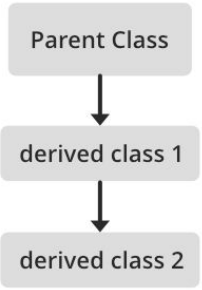
- Inheritance allows one class to acquire properties and methods of another class.
- Allows **reusability**



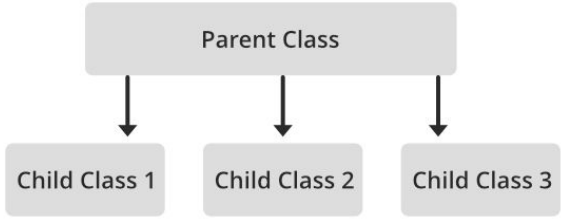
Single inheritance



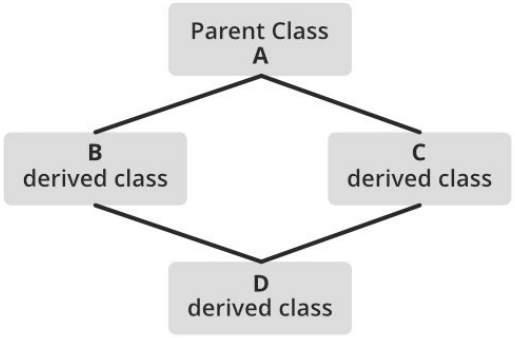
Multiple inheritance



Multilevel inheritance



Hierarchical inheritance



Hybrid Class

Polymorphism

- It is the concept by which a method or object takes different forms in different. environments.
  - E.g., the operator + adds two whole numbers, two decimal point numbers, strings.
    - Symbol * (star) shows polymorphic behavior.
    - It is used to multiply two values, incorporate comments such as /* and */, import a package into a program like

      import java.lang.*;.

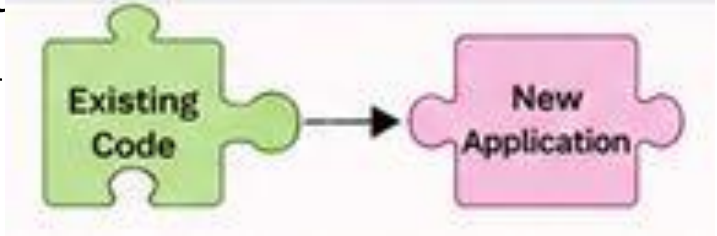
- Message Passing
  - Message passing means one object requests another object to perform an action by calling its method.





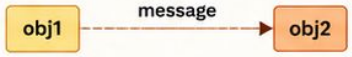
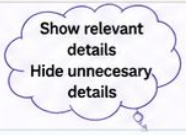
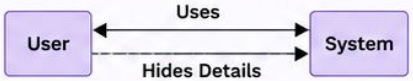
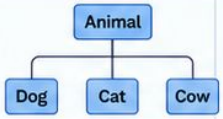
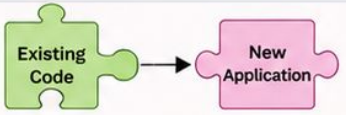
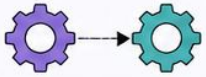
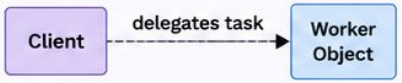
- Reusability
  - Code written once can be reused in many places. Inheritance and classes improve reusability.
  - Example: A Vehicle class can be reused by C Bus cl





- **Delegation**
  - Delegation means an object hands over a task to another object instead of doing it itself.
  - Computer delegates printing work to Printer.

Characteristics of Object-Oriented Programming (OOP)

Characteristic	Description	Illustration / Example
1 Class 	A class is a blueprint or template that defines the data (attributes) and behaviors (methods) that objects of the class will have.	`class Car { String brand; String color; void start(){ ... } void stop(){ ... } }`
2 Object 	An object is an instance of a class. It represents a real-world entity and has state (data) and behavior (methods).	`Car myCar = new Car();`  myCar
3 Methods and Message Passing 	Objects communicate with each other by sending messages. A message is a request to invoke a method.	`obj1.sendMessage(obj2);` 
4 Data Abstraction 	Data abstraction means showing only essential information to the user and hiding implementation details.	
5 Encapsulation and Data Hiding 	Binding data (attributes) and methods that operate on the data into a single unit (class) and restricting direct access to some of the object's components.	`class Account { private double balance; public void deposit(double amt){...} public double getBalance() {... } }`
6 Inheritance 	A mechanism in which a new class (acquired class) inherits properties and behaviors from an existing class (base class).	`class Dog extends Animal { void bark() { ... } }` 
7 Reusability 	OOP promotes reusability by allowing existing classes, methods, and components to be used in new programs.	
8 Polymorphism  Same operation, Different forms	The ability of an object to take many forms. The same method name can behave differently for different objects.	`shape.draw();` 
9 Delegation 	An object delegates a task to another object to do the work on its behalf.	

COHESION, COUPLING & REUSABILITY IN OOP

1. COHESION (Within a Class)

Cohesion is the degree to which the elements inside a class belong together.
High cohesion is desirable.

HIGH COHESION (Good)

Student

Data Members
- id
- name
- marks

Methods
- read()
- display()
- calculateGrade()

All data and methods are related to Student.

LOW COHESION (Bad)

Student

Data Members
- id
- name

Methods
- read()
- display()
- playMusic()
- calculateSalary()
- sendEmail()

Methods are unrelated to Student.

2. COUPLING (Between Classes)

Coupling is the degree of inter-dependence between the pairs of classes.
Low coupling is desirable.

LOW COUPLING (Good)

Student

Data Members
- id
- name
- marks

Methods
- display()
- printReport()

uses

Printer

Methods
- print()

Student class uses Printer class but both classes have their own responsibilities.

✓ LOW COUPLING

HIGH COUPLING (Bad)

Student

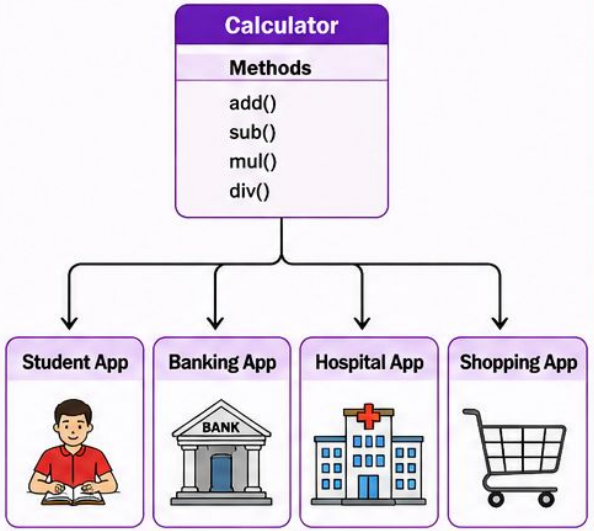
Methods
- read(), display(), print(), scan(), connectInternet(), playMusic(), sendEmail(), calculateSalary()

Too many responsibilities and dependencies in one class.

✕ HIGH COUPLING

3. REUSABILITY

Reusability is the ability to reuse the previously defined classes and objects.



The same class (Calculator) is reused in many applications.

✓ REUSABILITY

KEY POINTS

- **Cohesion** → Within a class (Keep related things together)
- **Coupling** → Between classes (Keep dependency low)
- **Reusability** → Use the same class again and again

THE GOLDEN RULE

HIGH COHESION + LOW COUPLING = GOOD DESIGN

BENEFITS

- Easy Maintenance
- Less Error
- Better Understandability
- Extensibility
- Reusability

C vs JAVA – 6 Key Differences

#	Aspect	C	JAVA
1	Approach	**Procedure-Oriented** Focuses on functions/procedures. Program is divided into functions.	**Object-Oriented** Focuses on objects and classes. Program is divided into objects.
2	Language Level	**Middle-Level Language** Provides a mix of high-level and low-level features.	**High-Level Language** Closer to human language and easy to understand and maintain.
3	Platform Dependency	**Platform-Dependent** Executable works only on the specific OS for which it is compiled.	**Platform-Independent** Compiled byte code runs on any OS using JVM (Write Once, Run Anywhere).
4	Memory Management	**Manual Memory Management** Programmer allocates and deallocates memory using malloc() and free().	**Automatic Memory Management** Memory is managed automatically by Garbage Collector.
5	Pointers	**Supports Pointers** Allows direct access to memory and pointer arithmetic.	**Does Not Support Pointers** Does not allow direct memory access (no pointer arithmetic).
6	Inheritance	**Does Not Support Inheritance** C is not an object-oriented language. No support for inheritance.	**Supports Inheritance** Java is a pure object-oriented language. Supports inheritance through classes.

Summary

C is suitable for system programming, hardware related tasks and performance-critical applications.

Java is suitable for large-scale, secure, platform-independent and enterprise applications.

Differences between C and JAVA

Parameter	C	JAVA
Language Level	Middle-level language	High-level language
Programming Paradigm	Structural, procedural	Object-oriented
Approach	Top-down	Bottom-up
Compilation	Compiled language	Compiled and Interpreted language
Platform Dependency	Platform dependent	Platform-independent (uses Java Virtual Machine)
Memory Management	Manual (functions like malloc, calloc, free)	Automatic (garbage collection)
Threading	Does not support threading	Supports multi-threading
Pointers	Supports pointers	Does not support pointers

Inheritance	Does not support inheritance	Supports inheritance
Error Handling	No built-in exception handling	Supports exception handling
File Extensions	.c	.java
Security	Less secure	More secure (due to lack of pointers, etc.)
Robustness	Less robust	More robust (strong memory management)
Supports	Applet concept is not there	Applets are there
Applications	System applications, embedded systems	Enterprise applications, web applications
Popularity	Less popular compared to Java	More Popular
Object-Oriented Programming (OOP) Features	Lacks OOP concepts like polymorphism, encapsulation	Fully supports OOP concepts

Diff between C++ and JAVA

Comparison Index	C++	Java
Platform-independent	C++ is platform-dependent.	Java is platform-independent.
Mainly used for	C++ is mainly used for system programming.	Java is mainly used for application programming. It is widely used in Windows-based, web-based, enterprise, and mobile applications.
Goto	C++ supports the goto statement.	Java doesn't support the goto statement.
Multiple inheritance	C++ supports multiple inheritance.	Java doesn't support multiple inheritance through class. It can be achieved by using interfaces in java.
Operator Overloading	C++ supports operator overloading.	Java doesn't support operator overloading.
Object-oriented	Partially	Fully

Pointers	C++ supports pointers. You can write a pointer program in C++.	Java supports pointer internally. However, you can't write the pointer program in java. It means java has restricted pointer support in java.
Compiler and Interpreter	C++ uses compiler only. C++ is compiled and run using the compiler which converts source code into machine code so, C++ is platform dependent.	Java uses both compiler and interpreter.
Call by Value and Call by reference	C++ supports both call by value and call by reference.	Java supports call by value only. There is no call by reference in java.
Structure and Union	C++ supports structures and unions.	Java doesn't support structures and unions.
Thread Support	C++ doesn't have built-in support for threads. It relies on third-party libraries for thread support.	Java has built-in thread support.

Tokens in Java Programs

- ***A Token is a individual/smallest unit of a program element which is recognized by the compiler.***
- ***A program contains set of tokens such as***
  - ***Identifiers or names*** This is meant for variables, methods, classes, etc.
  - ***Keywords*** These are special words defined in Java and represent a set of instructions.
  - ***Literals*** These are values represented by a set of characters, digits, etc.
  - ***Separators*** These include comma, semicolon, period (.), (), [], etc.
  - ***Operators*** - Operators are mostly represented by symbols such as + , −,*, etc.

Identifiers or Names

- Anything stored in the computer memory has a name or identifier which identifies that section of the memory
- Rules:

  1. It may be a single word or a combination of words with no space between words.
  2. It should not be a keyword of Java.
  3. It should not be Boolean literal, that is, true or false.
  4. It should not be **null** literal.
  5. It should not start with a digit but it can start with an underscore.
  6. It can comprise one or more unicode characters as well as digits.

Conventions for Writing Names

- Packages - Names of packages are completely in lower-case letters such as mypackage, java.lang.
- For Class - Names of classes and interfaces start with an upper-case letter.
- For Methods - Names of methods start with a lower-case character.
- For variables - Names of variables should start with a lower-case character.

Keywords in Java

- The keywords represent groups of instructions for the compiler.
- These are special tokens that have a predefined meaning and their use is restricted.
- keywords cannot be used as names of variables, methods, classes, or packages.
- These are written in the lower case.

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

JAVA KEYWORDS

Keywords are reserved words in Java that have a special meaning and cannot be used as identifiers (class, variable, method or object names).

53 KEYWORDS IN JAVA

1 DATA TYPES

byte short int
long float double
char boolean void

Used to declare variables and specify data types.

2 ACCESS MODIFIERS

public protected
private default

Used to control visibility and access to classes, methods and variables.

3 CONTROL FLOW

if for
else while
switch do return
case break
default continue

Used to control the flow of execution in a program.

4 CLASS & OBJECT

class new
interface this
extends super
implements

Used in the definition of classes, inheritance and object creation.

5 EXCEPTION HANDLING

try
catch throws
finally
throw

Used to handle exceptions and error management.

6 MODIFIERS

abstract transient
final volatile
static native
strictfp
synchronized

Used to modify classes, methods and variables.

7 PACKAGE RELATED

package instanceof
import

Used in package declaration, importing and type checking.

8 BOOLEAN LITERALS

true false

Used as boolean constant values.

9 LITERALS

null

Used to represent no reference.

10 RESERVED (FUTURE USE)

goto enum assert
const

Reserved for future use. (enum is used from Java 5 onwards.)

NOTE
These keywords cannot be used as identifiers.

TOTAL KEYWORDS: 53

Remember:
- Java is case-sensitive.
- Keywords must be written in lowercase.
- They have predefined meaning in Java.

```
class Example {
    public static void main(String[] args) {
        int value = 10;
        if (value > 0) {
            System.out.println("Positive");
        }
    }
}
```

Keywords make Java programs powerful!

Literals in Java

- A literal is a constant value
- The value may be a number (either whole or decimal point number) or a sequence of characters which is called String literal, Boolean type, etc.
- Types of Literals:

  1. Integer literals - 6, 453, 34789
  2. Floating point literal - 3.14159, 567.78
  3. Boolean literal - two values—true or false
  4. Character literal - Characters are represented in single quotes such as ‘A’, ‘H’, ‘k’,
  5. String literal - “AP”, “Amaravati”, “Vijayawada”,
  6. Null literal - null

Separators

In Java, **Separators** (also called **punctuators**) are special symbols used to separate program elements such as statements, blocks, method parameters, and array indices. According to the Java Language Specification, Java has **12 separators**.

Separator	Name	Purpose	Example
( )	Parentheses	Enclose method parameters and expressions	sum(a, b)
{ }	Braces	Define a block of code or class body	{ int x = 10; }
[ ]	Brackets	Declare and access arrays	int[] a; a[0]
;	Semicolon	Terminates a statement	int x = 10;

,	Comma	Separates variables, parameters, or arguments	int a, b, c;
.	Dot (Period)	Access members of a class/object	System.out.println()
...	Ellipsis	Variable-length arguments (varargs)	void show(int... n)
@	At Symbol	Introduces annotations	@Override
::	Method Reference Operator	References a method	System.out::println

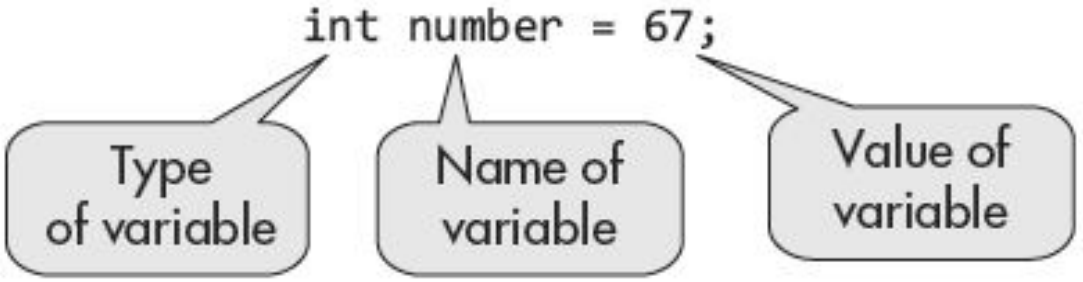
```
class Demo {
    public static void main(String[]
args) {
        int[] a = {10, 20, 30};

        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Separator	Usage
{ }	Class, method, and loop blocks
( )	Method declaration, main(), for(), println()
[ ]	Array declaration and indexing
;	Ends statements
.	Accesses System.out.println
,	Separates values in array

Declaration of Variables

- Variables are objects which contains some values, those values may change in the program.
- A variable is declared by first writing its type, [illegible] r.



JAVA STATEMENTS

A Java program consists of a set of instructions.
These instructions are called **statements**.

TYPES OF JAVA STATEMENTS

1. EMPTY STATEMENT

An empty statement contains only a semicolon (;).

It performs no action.

2. VARIABLE DECLARATION STATEMENT

Used to declare variables of different data types to store values.

3. LABELED STATEMENT

A statement preceded by an identifier and a colon (:).

Used with break and continue in loops.

4. EXPRESSION STATEMENT

Performs an operation.

Examples: assignment, method call, increment, object creation, etc.

5. CONTROL STATEMENTS

A. SELECTION STATEMENTS

Used for decision making.

Types:

- if
- if-else
- else-if ladder
- switch

B. ITERATION STATEMENTS

Used to repeat a block of code.

Types:

- while
- do-while
- for
- enhanced for (for-each)

C. JUMP STATEMENTS

Transfer control from one part to another.

Types:

- break
- continue
- return
- throw

6. SYNCHRONIZED STATEMENT

Used in multithreading to control access to shared resources.

Ensures that only one thread executes a block at a time.

7. GUARDING STATEMENTS (EXCEPTION HANDLING)

Used to handle runtime errors (exceptions) and maintain normal flow of the program.

Types:

- try
- catch
- finally
- throw / throws

JAVA STATEMENTS – TYPES AND CODE SNIPPETS

TYPES OF JAVA STATEMENTS

1 EMPTY STATEMENT



```
for(int i = 0; i < 5; i++);    // Empty statement ( ; ) acts as the loop body
```

2 VARIABLE DECLARATION STATEMENT



```
int age = 20;
double salary = 45000.50;
char grade = 'A';
```

3 LABELED STATEMENT

label:

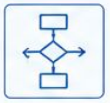
```
outer:
  for(int i = 1; i <= 3; i++) {
      for(int j = 1; j <= 3; j++) {
          if(i == 2)
              break outer;   // exit outer loop
      }
  }
```

4 EXPRESSION STATEMENT



```
x = 10;          // assignment      x++;          // increment
--x;             // decrement       display();    // method call
new Student();   // object creation
```

5 CONTROL STATEMENTS



A) SELECTION STATEMENTS

```
if(age >= 18)
   System.out.println("Eligible");
else
   System.out.println("Not Eligible");
```

B) ITERATION STATEMENTS

```
for(int i = 1; i <= 5; i++)
   System.out.println(i);

while(i < 5) {
   i++;
}
```

C) JUMP STATEMENTS

```
break;          // terminates loop/switch
continue;       // skips to next iteration
return;         // returns from method
throw new Exception(); // throws exception
```

6 SYNCHRONIZED STATEMENT



```
synchronized(this) {
    count++;
}
```

// only one thread
// can execute this
// block at a time

7 GUARDING STATEMENTS (EXCEPTION HANDLING)



```
try {
    int x = 10 / 0;
} catch(ArithmeticException e) {
    System.out.println(e);
} finally {
    System.out.println("Done");
}
```

// try: risky code
// catch: handles exception
// finally: always executed

CODE SNIPPETS (For Understanding)



Note: Every Java program is built using one or more of the above statements.

These statements perform different actions and together control the flow of execution in a program.

Java Statements

- A Java program consists of a set of instructions, these instructions are the statements in Java.
- Empty statement –
  - An empty statement in Java is written as a single semicolon.
  - For example, you can use it to indicate an empty loop body of a for loop:

```
for(int i = 0; i < 10; a[i++]++) // Increment array elements
    /* empty */; // Loop body is empty statement
```

- Variable declaration statements – a

- Labelled statements
  A labeled statement is simply a statement that has been given a name by prepending a identifier and a colon to it.
  Labels are used by the break and continue statements.

```
rowLoop: for(int r = 0; r < rows.length; r++)
{
      // A labeled loop
            colLoop: for(int c = 0; c < columns.length; c++)
            {
                  // Another one break rowLoop;
                  // Use a label
             }
}
```

- Expression statements:
  var=expr; expr++; ++expr; method(); newType(); --expr; expr- -;
- Control statements
  - Selection statements
  - Iteration statements
  - Jump statements - break, continue, return and throw
- Synchronized statement
- Guarding statement – Exceptions - try, catch block and finally

Command line arguments

- A Java application can accept any number of arguments from the command line.
- These arguments can be passed at the time of running the program.
- length property array is used to find number of arguments

User input to programs

- There are various ways to read input from the keyboard, the java.util.Scanner class is one of them.
- The **Java Scanner** class breaks the input into tokens using a delimiter that is whitespace by default.
- It provides many methods to read and parse various primitive values.
- Part of util package.
- Programs discussed in text book:
  - Performing arithmetic operations on two numbers
  - Finding Area of a circle

Method	Description
public String next()	it returns the next token from the scanner.
public String nextLine()	it reads the given input as one line irrespective of white space and returns the value as a string.
public byte nextByte()	it scans the next token as a byte.
public short nextShort()	it scans the next token as a short value.
public int nextInt()	it scans the next token as an int value.
public long nextLong()	it scans the next token as a long value.
public float nextFloat()	it scans the next token as a float value.
public double nextDouble()	it scans the next token as a double value.

ESCAPE SEQUENCES IN JAVA

Escape sequences are special characters that begin with a backslash (\).
They are used in strings and character literals for formatting and special characters.

ESCAPE SEQUENCE	DESCRIPTION	EXAMPLE	OUTPUT
\n	New line	"Hello\nJava"	Hello Java
\t	Horizontal tab	"Java\tProgramming"	Java Programming
\b	Backspace (deletes the previous character)	"ABC\bD"	ABD
\r	Carriage return (moves cursor to the beginning of line)	"Hello\rJava"	Java *(overwrites the beginning)*
\f	Form feed (page break)	"Hello\fWorld"	Hello --- World *(form feed character)*
\'	Single quotation mark	'\''	'
\"	Double quotation mark	"He said, \"Hi\""	He said, "Hi"
\\	Backslash	"C:\\Java"	C:\Java

UNICODE Escape sequences

- \uXXXX represents a **Unicode escape sequence** in Java, where XXXX is a **4-digit hexadecimal Unicode value**.
- Unicode escape sequences enable Java programs to display international currency symbols, mathematical symbols, and characters from almost every language.
- The **Indian Rupee symbol (₹)** is represented by **\u20B9** and is supported in Java 7 and later.

UNICODE Escape Characters

Char	Unicode	Escape Sequence	Description
&	U+0026	\u0026	Ampersand
•	U+2022	\u2022	Bullet
	U+25E6	\u25E6	White Bullet
⋅	U+2219	\u2219	Bullet Operator
‣	U+2023	\u2023	Triangular Bullet
⁃	U+2043	\u2043	Hyphen Bullet
°	U+00B0	\u00B0	Degree
∞	U+221E	\u221E	Infinity

$	U+0024	\u0024	Dollar
€	U+20AC	\u20AC	Euro
£	U+00A3	\u00A3	Pound
¥	U+00A5	\u00A5	Yen / Yuan
¢	U+00A2	\u00A2	Cent
₹	U+20B9	\u20B9	Indian Rupee
Rs	U+20A8	\u20A8	Rupee
₱	U+20B1	\u20B1	Peso
₩	U+20A9	\u20A9	Korean Won
฿	U+0E3F	\u0E3F	Thai Baht
₫	U+20AB	\u20AB	Dong
₪	U+20AA	\u20AA	Shekel

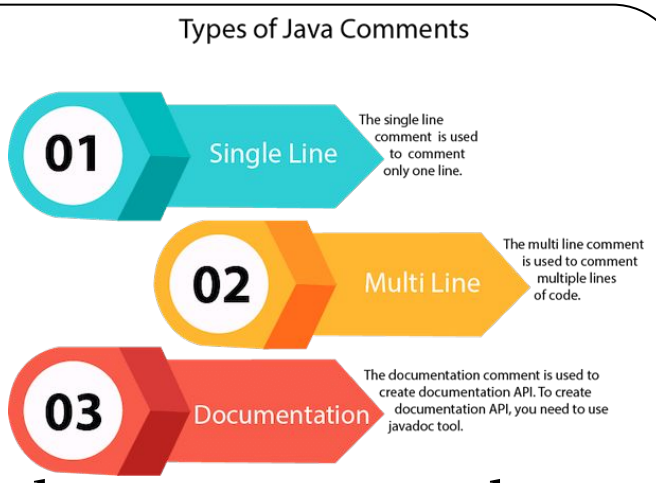
Intellectual Property Codes			
©	U+00A9	\u00A9	Copyright
®	U+00AE	\u00AE	Registered Trademark
℗	U+2117	\u2117	Sound Recording Copyright
™	U+2122	\u2122	Trademark
℠	U+2120	\u2120	Service Mark

Octal Escape Sequence in Java

- An **Octal Escape Sequence** represents a character using its **octal (base-8)** ASCII value.
- It begins with a **backslash (\)** followed by **1 to 3 octal digits i.e., 0-7**.

Decimal	Character	Java Escape – Octal Code
65	A	\101
66	B	\102
67	C	\103
90	Z	\132
97	a	\141
98	b	\142
99	c	\143
122	z	\172
48	0	\060
49	1	\061
50	2	\062
32	Space	\040

Comments



- 3 types
- Single Line Comment
  - The single-line comment is used to comment only one line of the code.
  - // Not executed
- Multiline Comment -/* and */
- Documentation comments are usually used to write large programs for a project or software application as it helps to create documentation API. These APIs are needed for reference, i.e., which classes, methods, arguments, etc., are used in the code.
  - To create documentation API, we need to use the javadoc tool. The documentation comments are placed between /** and */.

Doc Comments

```
F:\Courses\JAVA26-27\UNIT 1 PPT and Material\Programs>javac DocumentationComment.java

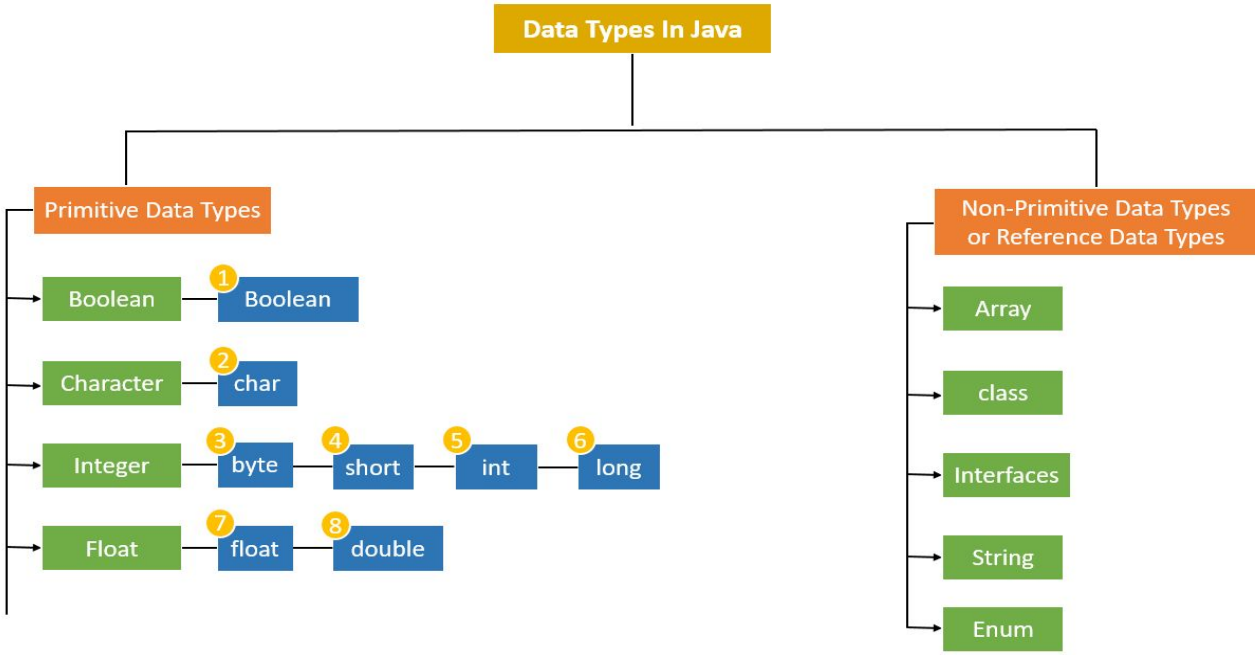
F:\Courses\JAVA26-27\UNIT 1 PPT and Material\Programs>java DocumentationComment
Addition of numbers: 60

F:\Courses\JAVA26-27\UNIT 1 PPT and Material\Programs>javadoc DocumentationComment.java
Loading source file DocumentationComment.java...
Constructing Javadoc information...
```

Data Types in Java

- Java defines eight primitive (basic) types of data
- 1. Integral types—**byte, short, int, long**
- 2. Floating point types—**float, double**
- 3. Character type—**char**
- 4. Boolean type—**boolean**

- There is a non-data type called **void**.
- This type is used for methods that do not return any value.



Declaration of Variables

- A variable is declared by first declaring its *type* followed by its identifier or name.
- The name or identifier should not be a keyword of Java language.
- It can be of any length and may consist of characters and digits.



Non-primitive Types

- These are the class and interface types. The name of a class or interface is the name of type. A class object is declared as

```
Class_identifier object_identifier;
```

Integers

- Integers are whole numbers, that is, they represent numbers that do not have a fractional part.
- In Java, these numbers may be positive or negative.

Table 3.1 Integer types and ranges

Type name	*Memory allocated in bytes*	*Range of values of variables*
byte	1 byte (8 bits)	–128 to 127
short	2 bytes (16 bits)	–32,768 to 32,767
int	4 bytes (32 bits)	–2,147,483,648 to 2,147,483,647
long	8 bytes (64 bits)	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

Characters

- Java supports Unicode for the representation of characters.
- Unicode supports all the character sets of all the major languages of the world.
- The initial version of Unicode allocated 2 bytes for storing characters.
- The range of values for characters in the initial version of Unicode comprised from ‘\u0000’ to ‘\uffff ’ that is from 0 to 65535 both end values inclusive.
  - ‘A’ - \u0041 – 65
  - ‘1’ - \u0031 - 49

Floating-point numbers

- The numbers that are not whole numbers, or those that have fractional part, Examples are 3.141, 476.6, and so on.
- Java supports two types of such numbers.
- **Float:** This type is used for single-precision decimal floating point numbers, that is, 7(or 8) digits after the decimal point.
- These numbers are stored in 4 bytes .
- **Double** This type is used for double-precision decimal floating point numbers, that is, 15 digits after the decimal point.

The letter f or F is appended to float value otherwise number will be considered as double value

Table 3.3 Range of values of `float` and `double` type variables

Data Type	*Memory allocated*	*Range of values*
`float`	4 byte	1.4e – 45 to 3.4e + 38
`double`	8 byte	4.9e – 324 to 1.8e + 308

1.4E-45 is **scientific notation (exponential notation)**.

It means: 1.4×10^{-45}

E means **"× 10 raised to the power of"**.
E+ means move the decimal point to the **right**.
E- means move the decimal point to the **left**.

POSITIVE EXPONENT (+)

Move the decimal point to the RIGHT.

Expression	Meaning	How to Move	Result
2.5E3	2.5×10^{3}	Move RIGHT 3 places	2.5 → 25.0 → 250. → 2500. = 2500
2.5E2	2.5×10^{2}	Move RIGHT 2 places	2.5 → 25.0 → 250. = 250
2.5E1	2.5×10^{1}	Move RIGHT 1 place	2.5 → 25.0 = 25
2.5E0	2.5×10^{0}	No Movement	2.5 = 2.5

NEGATIVE EXPONENT (–)

Move the decimal point to the LEFT.

Expression	Meaning	How to Move	Result
2.5E-1	2.5×10^{-1}	Move LEFT 1 place	2.5 → 0.25 = 0.25
2.5E-2	2.5×10^{-2}	Move LEFT 2 places	2.5 → 0.25 → 0.025 = 0.025
2.5E-3	2.5×10^{-3}	Move LEFT 3 places	2.5 → 0.25 → 0.025 → 0.0025 = 0.0025